The matplotlib package

Ezequiel López-Rubio

Department of Computer Languages and Computer Science

University of Málaga, Spain

Contents

- Introduction
- Fundamentals
- Figure customization
- Conclusion

1. INTRODUCTION

1. Introduction

Overview

- The matplotlib package is a standard library for creating visualizations in Python
- The usual way to import the package is:

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

- For Google Colab the following line must be added:

```
%matplotlib inline
```

- The package graphs the desired data on Figure objects
  - Each figure contains one or more Axes objects

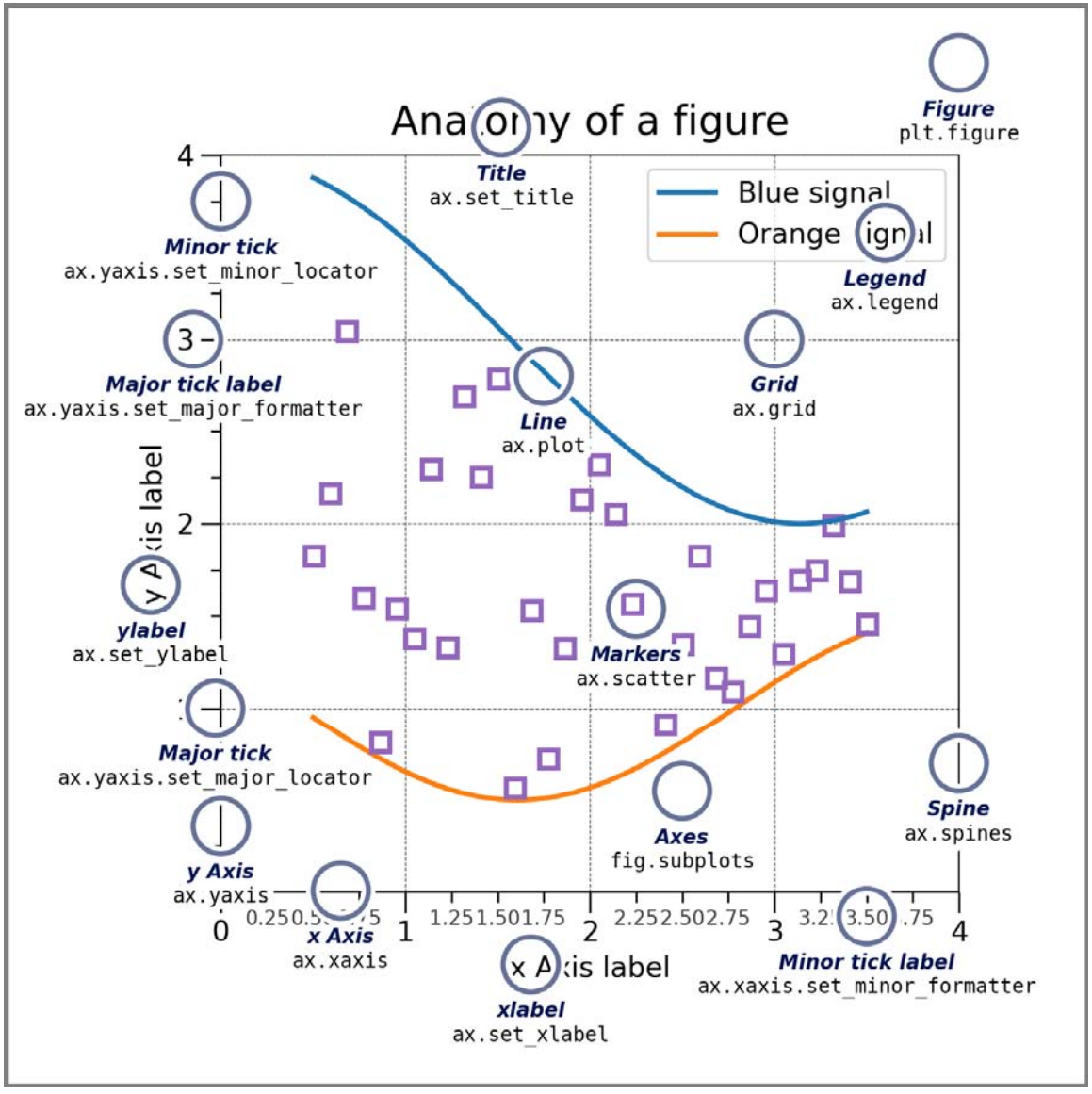
1. Introduction

Class structure

- The Figure object represents the whole figure
  - The Figure keeps track of all the child Axes, a group of 'special' Artists (titles, figure legends, colorbars, etc), and even nested subfigures
- An Axes object is an Artist attached to a Figure that contains a region for plotting data, and usually includes two (or three in the case of 3D) Axis objects
- Each Axis object provides ticks and tick labels to provide scales for the data in the Axes
- Basically, everything visible on the Figure is an Artist (even Figure, Axes, and Axis objects). This includes Text objects, Line2D objects, collections objects, Patch objects, etc.

1. Introduction

Anatomy of a figure



2. FUNDAMENTALS

2. Fundamentals

Types of inputs

- Plotting functions expect numpy.array as input, or objects that can be passed to numpy.as_array
- Classes that are similar to arrays ('array-like') such as pandas data objects and numpy.matrix may not work as intended
- Common convention is to convert these to numpy.array objects prior to plotting
- Most methods will also parse an addressable object like a dictionary, or a pandas.DataFrame
  - Matplotlib allows you to provide the data keyword argument and generate plots passing the strings corresponding to the x and y variables

2. Fundamentals

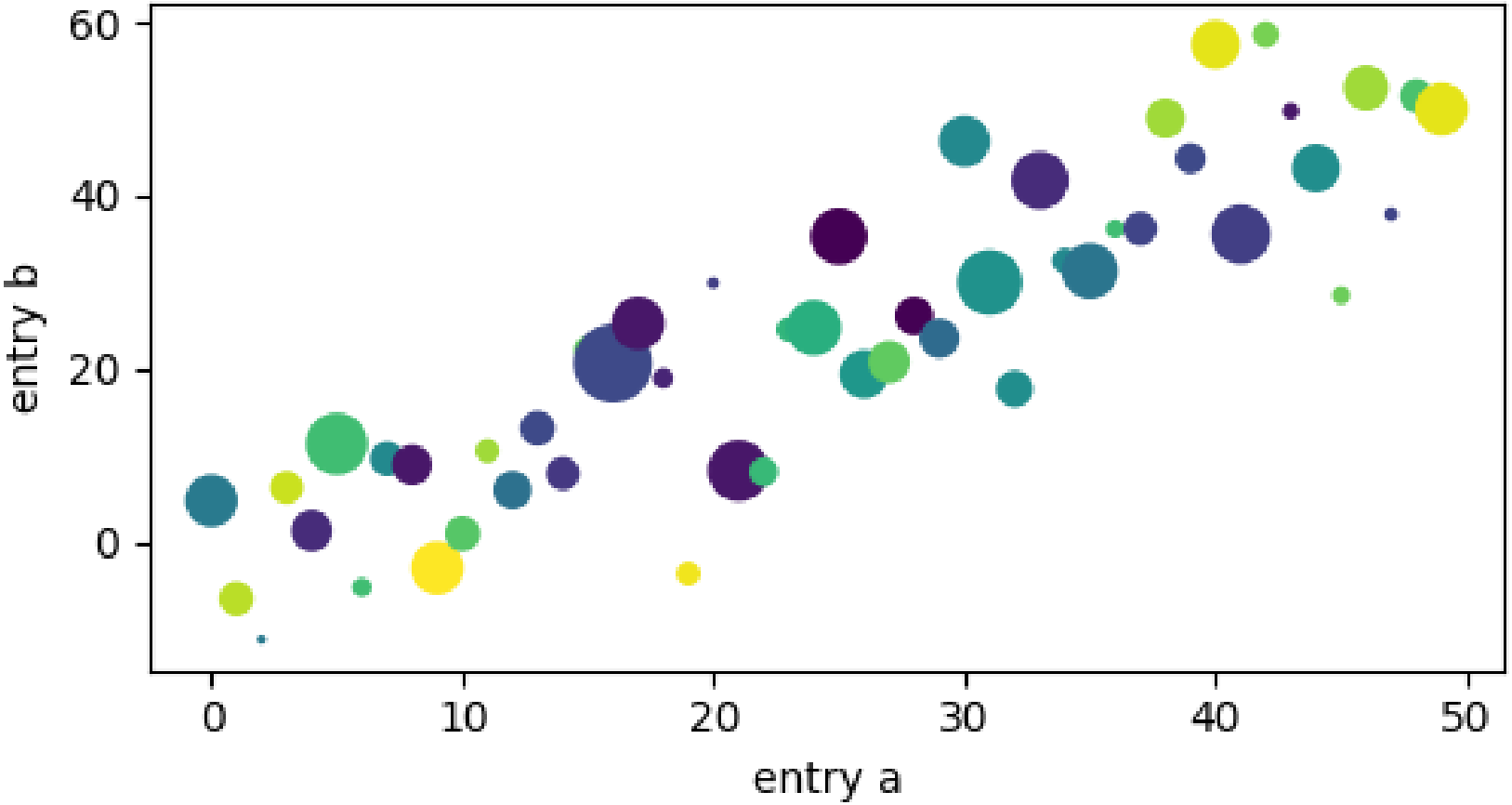
Addressable object input example

```
np.random.seed(19680801)  # seed the random number generator
data = {'a': np.arange(50),
        'c': np.random.randint(0, 50, 50),
        'd': np.random.randn(50)}
data['b'] = data['a'] + 10 * np.random.randn(50)
data['d'] = np.abs(data['d']) * 100

fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.scatter('a', 'b', c='c', s='d', data=data)
ax.set_xlabel('entry a')
ax.set_ylabel('entry b')
```

2. Fundamentals

Addressable object input example



2. Fundamentals

Coding styles

- Matplotlib has two major application interfaces, or coding styles of using the library:
  - An explicit "Axes" interface that uses methods on a Figure or Axes object to create other Artists, and builds a visualization step by step. This has also been called an "object-oriented" interface
  - An implicit "pyplot" interface that keeps track of the last Figure and Axes created, and adds Artists to the object it thinks the user wants
- The "Axes" interface is how Matplotlib is implemented, and many customizations and fine-tuning end up being done at this level
- The pyplot module shadows most of the Axes plotting methods to give the equivalent of the above, where the creation of the Figure and Axes is done for the user

2. Fundamentals

Example of the explicit interface

```
>>> x = np.linspace(0, 2, 100)  # Sample data.
>>> # Note that even in the OO-style, we use `.pyplot.figure` to create the Figure
>>> fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
>>> ax.plot(x, x, label='linear')  # Plot some data on the axes
>>> ax.plot(x, x**2, label='quadratic')  # Plot more data on the axes...
>>> ax.plot(x, x**3, label='cubic')  # ... and some more
>>> ax.set_xlabel('x label')  # Add an x-label to the axes
>>> ax.set_ylabel('y label')  # Add a y-label to the axes
>>> ax.set_title("Simple Plot")  # Add a title to the axes
>>> ax.legend()  # Add a legend
```

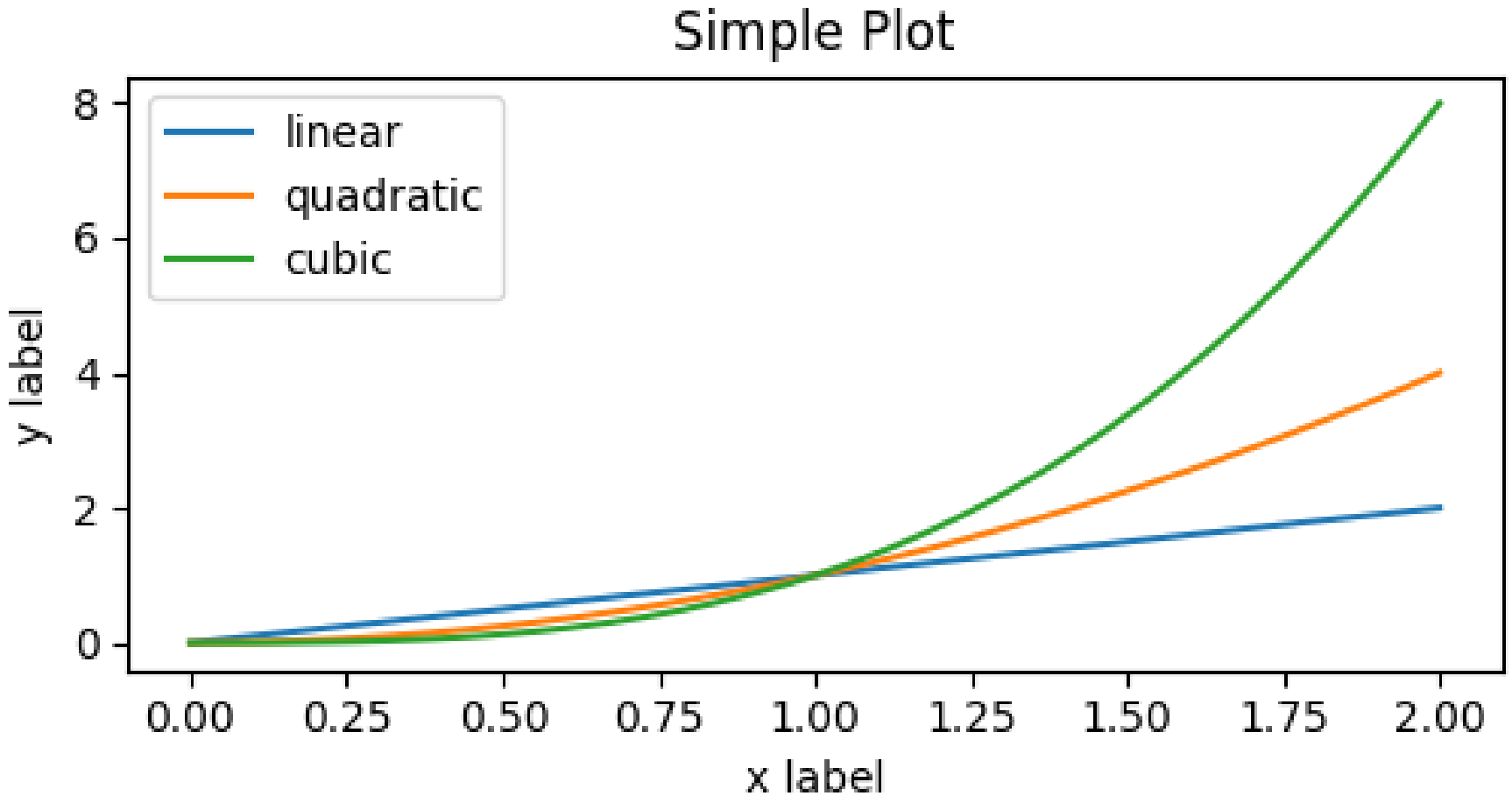
2. Fundamentals

Example of the implicit interface

```
>>> x = np.linspace(0, 2, 100)  # Sample data.
>>> plt.figure(figsize=(5, 2.7), layout='constrained')
>>> plt.plot(x, x, label='linear')  # Plot data on the (implicit) axes
>>> plt.plot(x, x**2, label='quadratic')  # etc
>>> plt.plot(x, x**3, label='cubic')
>>> plt.xlabel('x label')
>>> plt.ylabel('y label')
>>> plt.title("Simple Plot")
>>> plt.legend()
```

2. Fundamentals

Result of both examples



2. Fundamentals

Exporting figures

- Figures can be exported with plt.savefig
- A file path or a Python file-like object should be specified in the fname parameter
- The output file format should be specified with the format parameter: 'png', 'pdf', 'svg', ...
- The dpi argument can be set to specify the dots per inch
- The transparent argument can be set to control the transparency of the figure

3. FIGURE CUSTOMIZATION

3. Figure customization

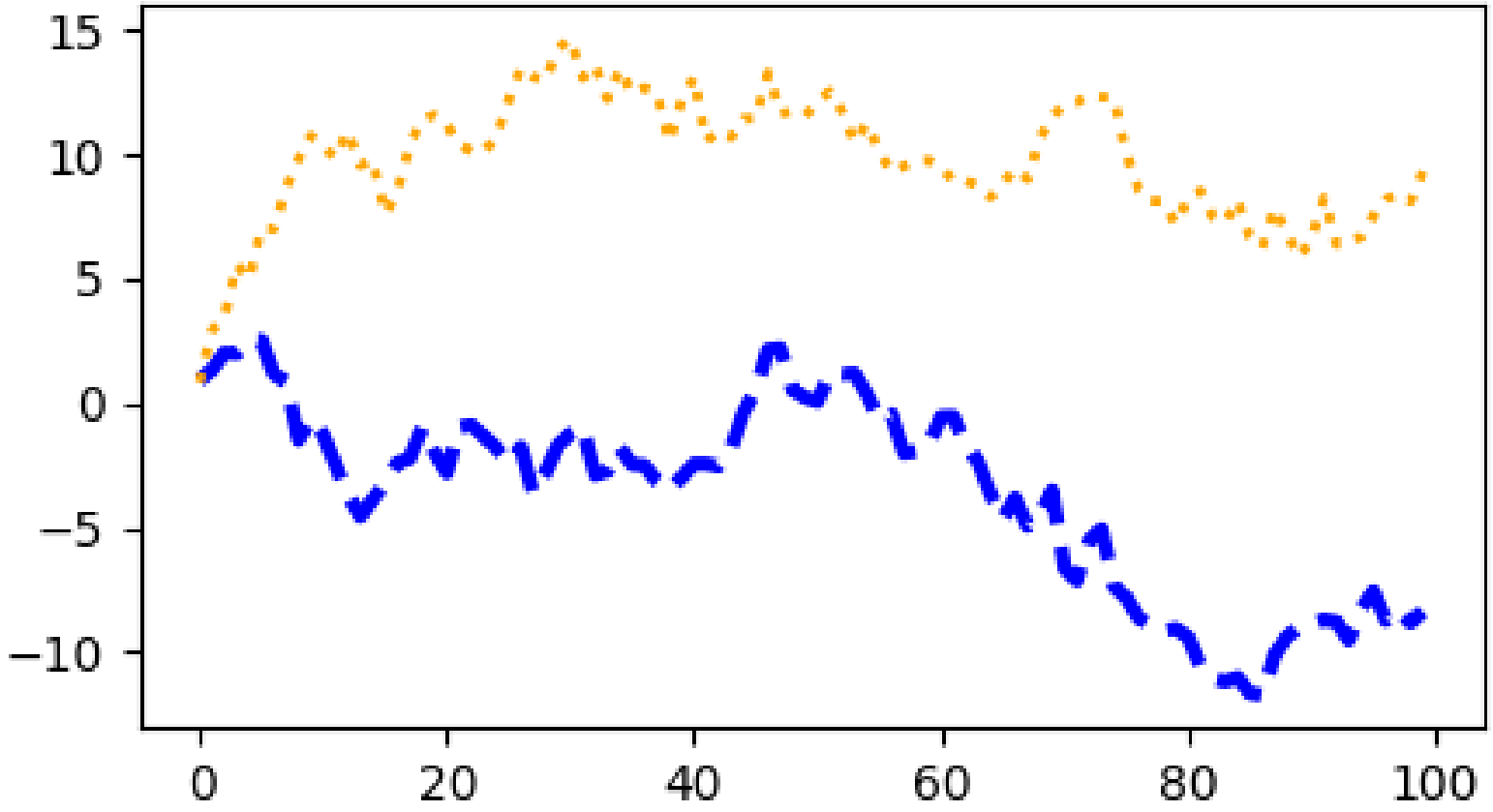
Styling artists

- Most plotting methods have styling options for the Artists, accessible either when a plotting method is called, or from a "setter" on the Artist
- In the example below we manually set the color, linewidth, and linestyle of the Artists created by plot, and we set the linestyle of the second line after the fact with set_linestyle

```
>>> data1, data2, data3, data4 = np.random.randn(4, 100)
>>> fig, ax = plt.subplots(figsize=(5, 2.7))
>>> x = np.arange(len(data1))
>>> ax.plot(x, np.cumsum(data1), color='blue', linewidth=3, linestyle='--')
>>> l, = ax.plot(x, np.cumsum(data2), color='orange', linewidth=2)
>>> l.set_linestyle(':')
```

3. Figure customization

Styling artists



3. Figure customization
Colors, linewidths, linestyles, and markersizes

- Some Artists will take multiple colors, i.e., for a scatter plot, the edge of the markers can be different colors from the interior

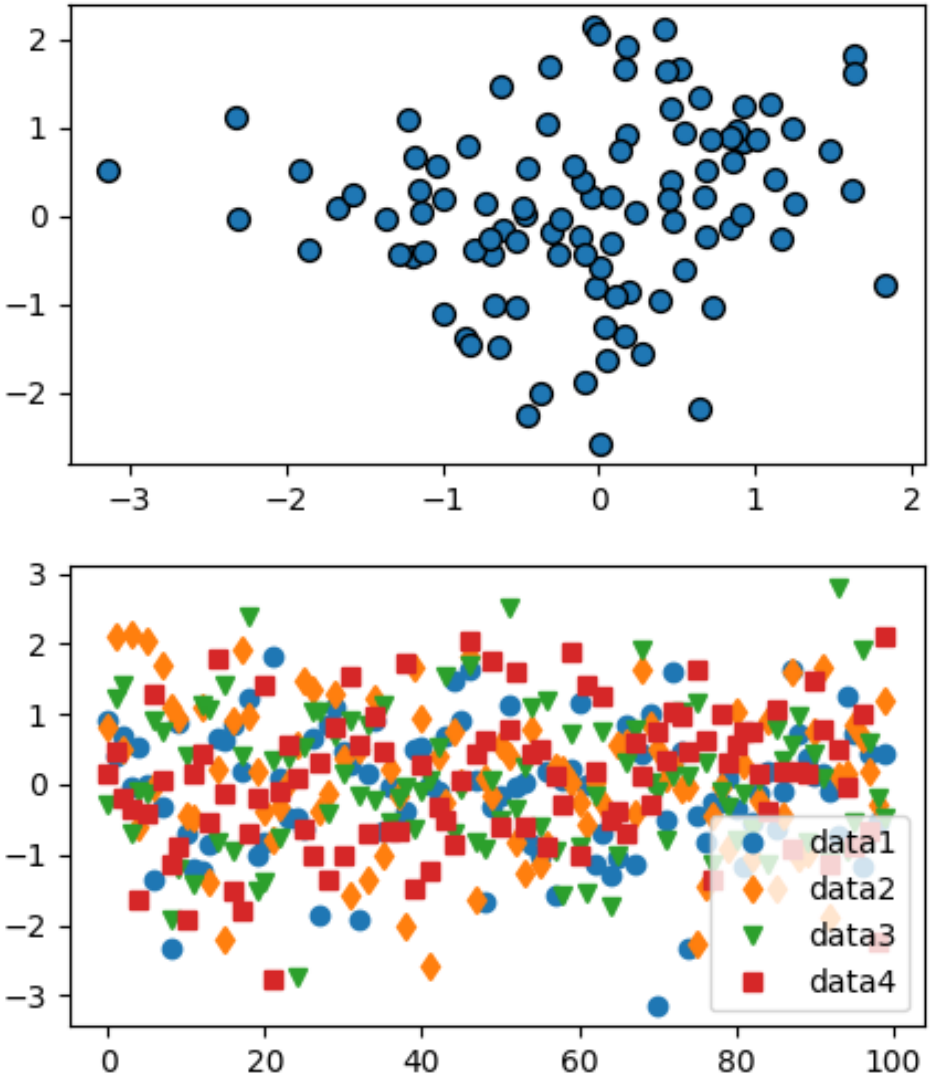
```
>>> fig, ax = plt.subplots(figsize=(5, 2.7))
>>> ax.scatter(data1, data2, s=50, facecolor='C0', edgecolor='k')
```

- Line widths are available for Artists that have stroked lines. Similarly, stroked lines can have a linestyle
- Marker size depends on the method being used. There is an array of markerstyles available as string codes

```
>>> fig, ax = plt.subplots(figsize=(5, 2.7))
>>> ax.plot(data1, 'o', label='data1')
>>> ax.plot(data2, 'd', label='data2')
>>> ax.plot(data3, 'v', label='data3')
>>> ax.plot(data4, 's', label='data4')
>>> ax.legend()
```

3. Figure customization

Colors, linewidths, linestyles, and markersizes



3. Figure customization

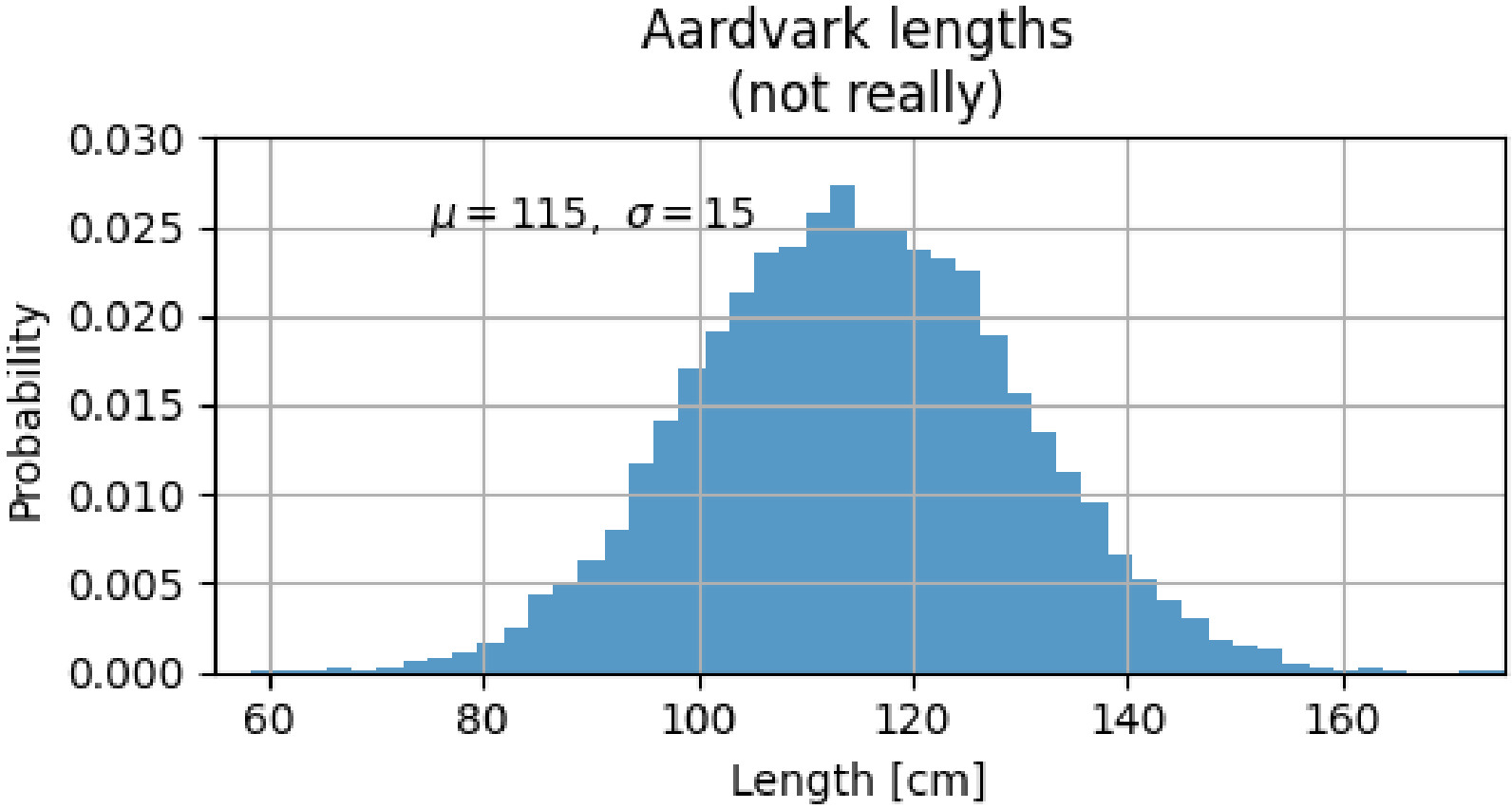
Axes labels and text

- Functions set_xlabel, set_ylabel, and set_title (from mpl.axes.Axes) are used to add text in the indicated locations. Text can also be directly added to plots using mpl.axes.Axes.text:

```
>>> mu, sigma = 115, 15
>>> x = mu + sigma * np.random.randn(10000)
>>> fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
>>> # the histogram of the data
>>> n, bins, patches = ax.hist(x, 50, density=True, facecolor='C0', alpha=0.75)
>>> ax.set_xlabel('Length [cm]')
>>> ax.set_ylabel('Probability')
>>> ax.set_title('Aardvark lengths\n (not really)')
>>> ax.text(75, .025, r'$\mu=115,\ \sigma=15$')
>>> ax.axis([55, 175, 0, 0.03])
>>> ax.grid(True)
```

3. Figure customization

Axes labels and text



3. Figure customization

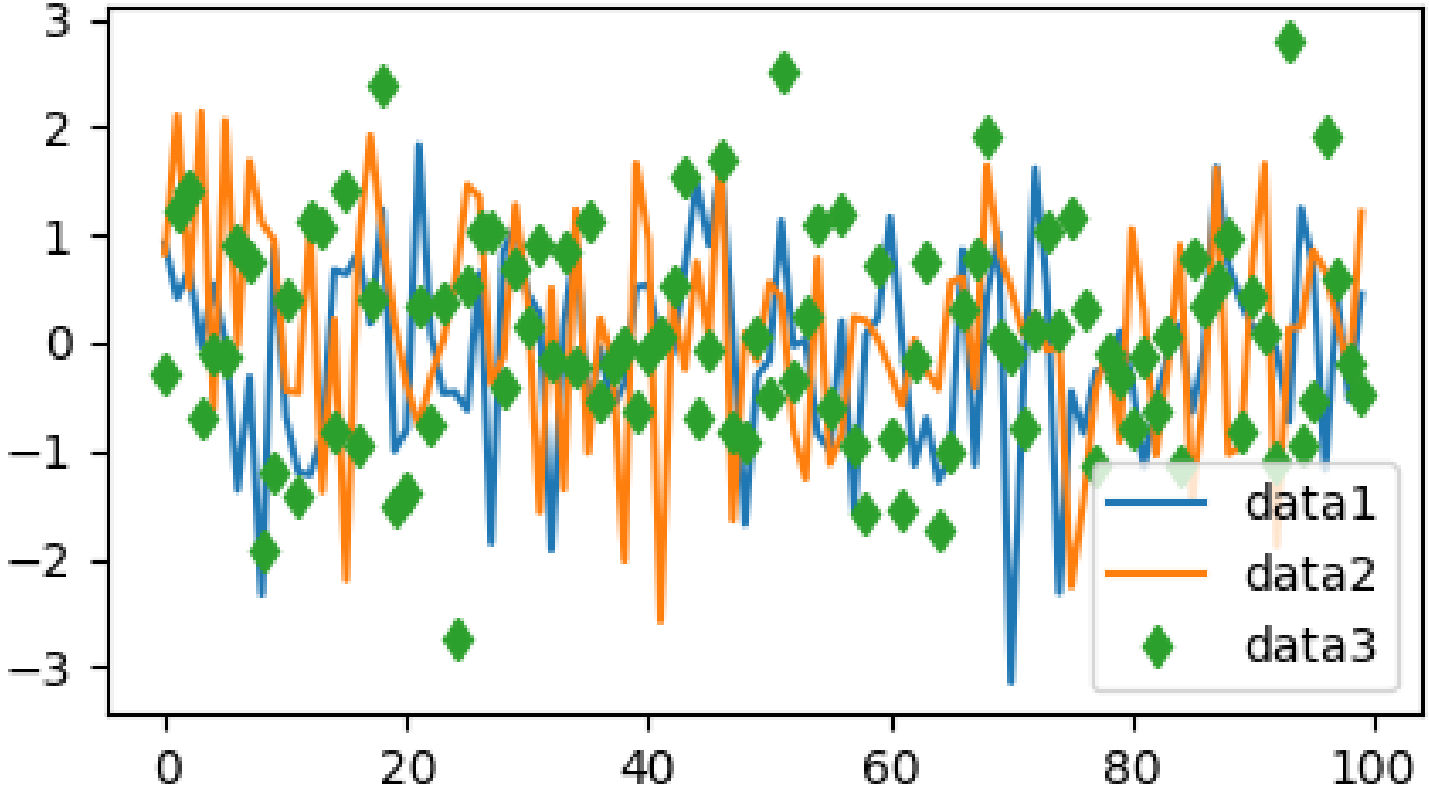
Legends

Often, we want to identify lines or markers with a Axes.legend:

```
>>> fig, ax = plt.subplots(figsize=(5, 2.7))
>>> ax.plot(np.arange(len(data1)), data1, label='data1')
>>> ax.plot(np.arange(len(data2)), data2, label='data2')
>>> ax.plot(np.arange(len(data3)), data3, 'd', label='data3')
>>> ax.legend()
```

3. Figure customization

Legends



3. Figure customization

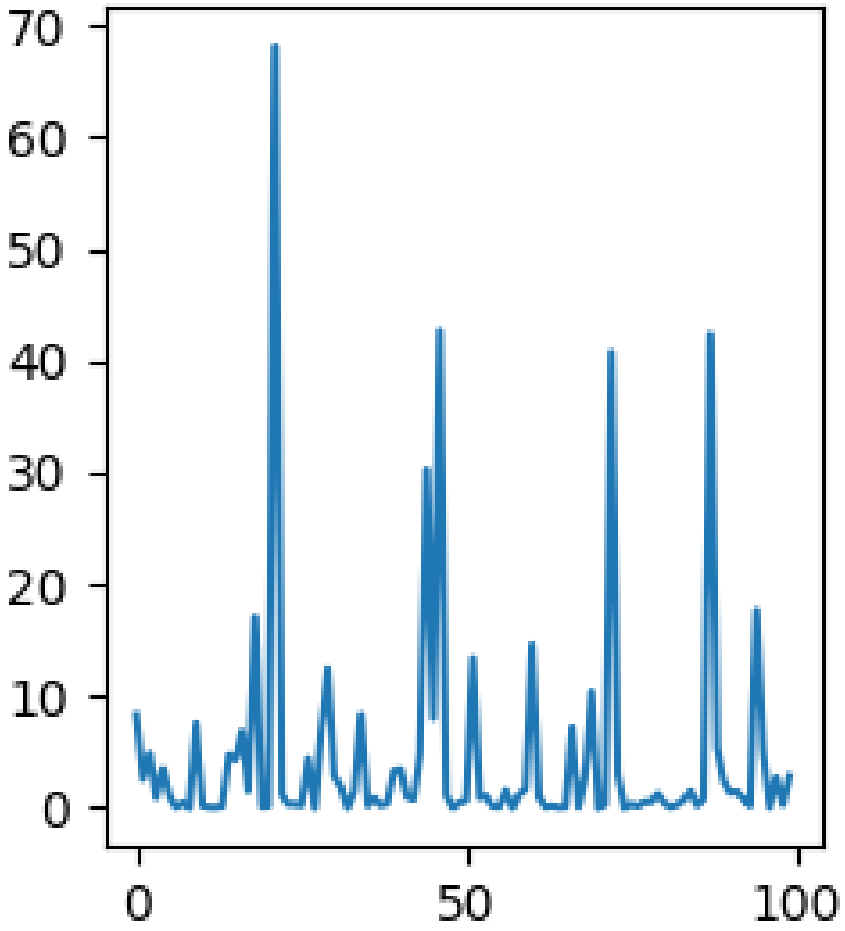
Axis scales

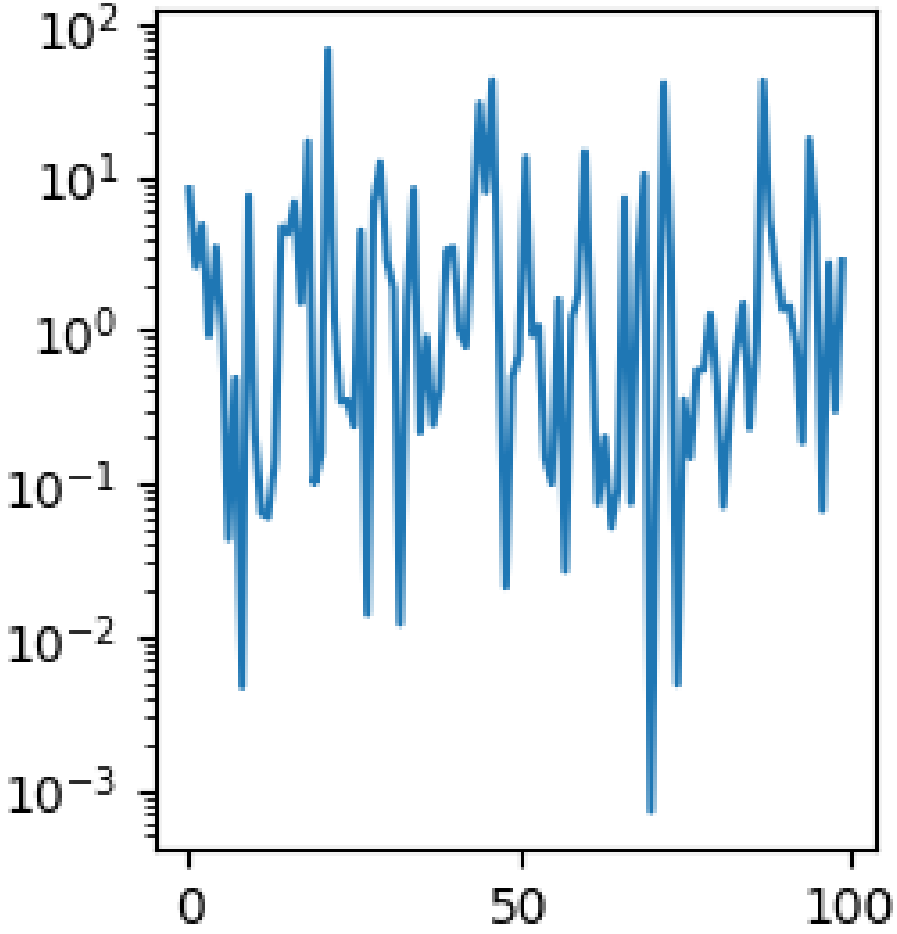
- Each Axes has two (or three) Axis objects representing the x- and y-axis. These control the scale of the Axis, the tick locators and the tick formatters
- In addition to the linear scale, Matplotlib supplies non-linear scales, such as a log-scale. Since log-scales are used so much there are also direct methods like loglog, semilogx, and semilogy. There are a number of scales

```
>>> fig, axs = plt.subplots(1, 2, figsize=(5, 2.7), layout='constrained')
>>> xdata = np.arange(len(data1))  # make an ordinal for this
>>> data = 10**data1
>>> axs[0].plot(xdata, data)
>>> axs[1].set_yscale('log')
>>> axs[1].plot(xdata, data)
```

3. Figure customization

Axis scales





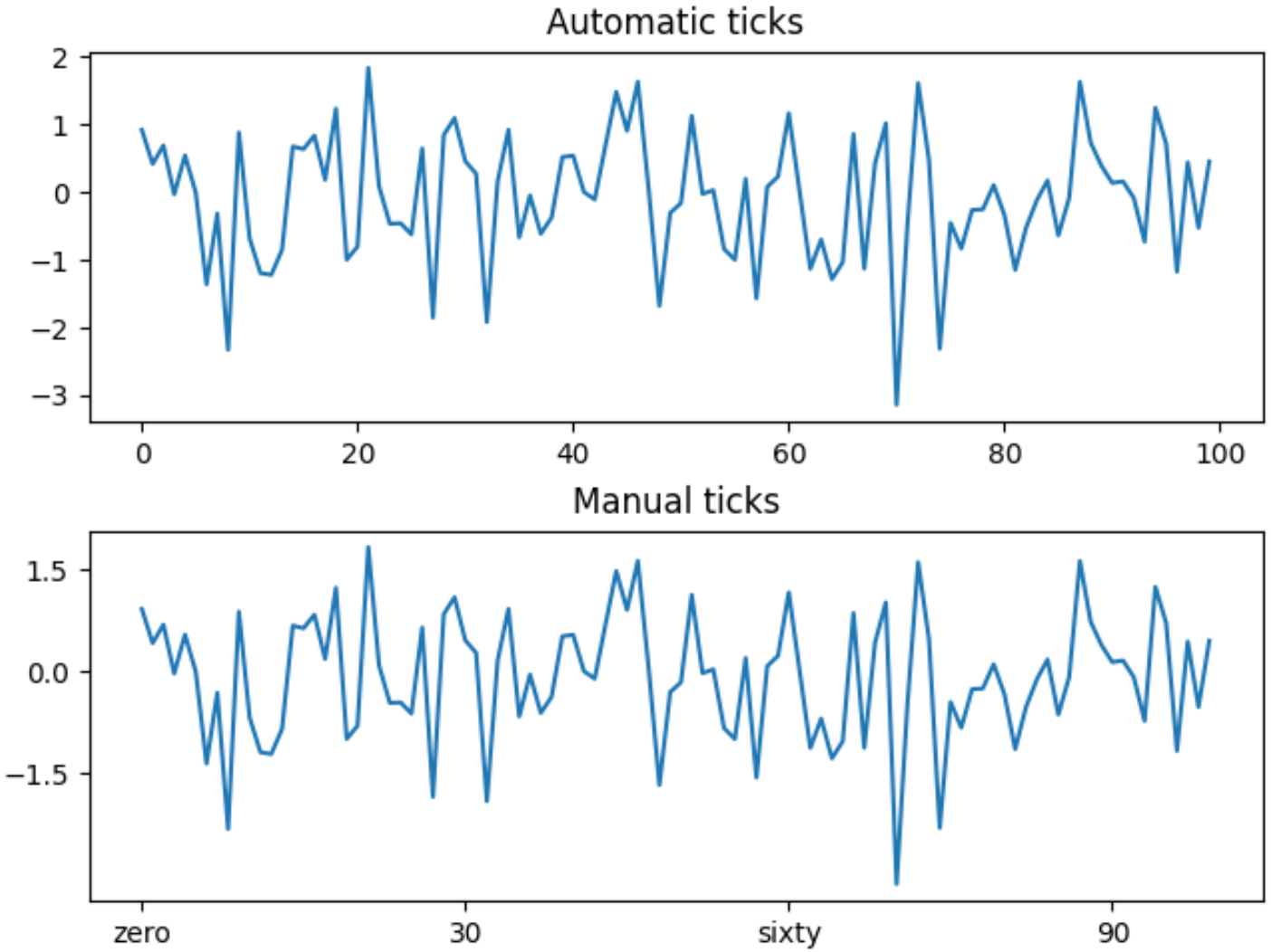
3. Figure customization
Tick locators and formatters

- Each Axis has a tick locator and formatter that choose where along the Axis objects to put tick marks. A simple interface to this is mpl.axes.Axes.set_xticks

```
>>> fig, axs = plt.subplots(2, 1, layout='constrained’)
>>> axs[0].plot(xdata, data1)
>>> axs[0].set_title('Automatic ticks’)
>>> axs[1].plot(xdata, data1)
>>> axs[1].set_xticks(np.arange(0, 100, 30), ['zero', '30', 'sixty', '90’])
>>> axs[1].set_yticks([-1.5, 0, 1.5])  # note that we don't need to specify labels
>>> axs[1].set_title('Manual ticks')
```

3. Figure customization

Tick locators and formatters



3. Figure customization

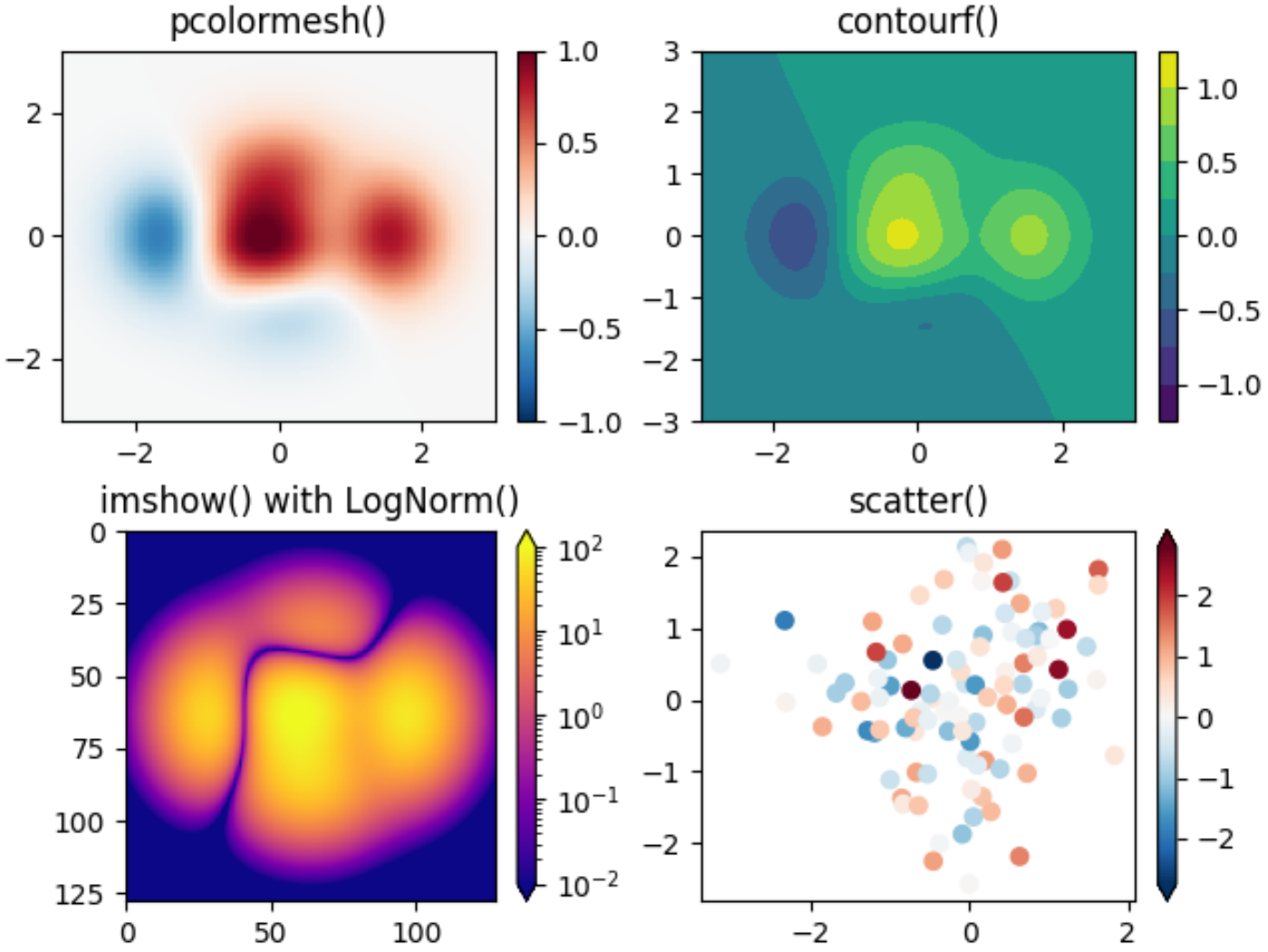
Color mapped data

- Often, we want to have a third dimension in a plot represented by colors in a colormap

```
>>> X, Y = np.meshgrid(np.linspace(-3, 3, 128), np.linspace(-3, 3, 128))
>>> Z = (1 - X/2 + X**5 + Y**3) * np.exp(-X**2 - Y**2)
>>> fig, axs = plt.subplots(2, 2, layout='constrained')
>>> pc = axs[0, 0].pcolormesh(X, Y, Z, vmin=-1, vmax=1, cmap='RdBu_r')
>>> fig.colorbar(pc, ax=axs[0, 0])
>>> axs[0, 0].set_title('pcolormesh()')
>>> co = axs[0, 1].contourf(X, Y, Z, levels=np.linspace(-1.25, 1.25, 11))
>>> fig.colorbar(co, ax=axs[0, 1])
>>> axs[0, 1].set_title('contourf()')
>>> pc = axs[1, 0].imshow(Z**2 * 100, cmap='plasma',
                          norm=mpl.colors.LogNorm(vmin=0.01, vmax=100))
>>> fig.colorbar(pc, ax=axs[1, 0], extend='both')
>>> axs[1, 0].set_title('imshow() with LogNorm()')
>>> pc = axs[1, 1].scatter(data1, data2, c=data3, cmap='RdBu_r')
>>> fig.colorbar(pc, ax=axs[1, 1], extend='both')
>>> axs[1, 1].set_title('scatter()')
```

3. Figure customization

Color mapped data



4. CONCLUSION

4. Conclusion

- The matplotlib package is a well-known standard library for data visualization in Python
- There are two coding styles available, depending on the degree of customization that is required
- The most used plot types are scatter plots, line plots, histograms, and color maps
- Many options are available to customize the figures
- Figures can be exported to a wide range of image file formats